

Adversarial Attacks

Robust Deep Learning (Seminar)

Bjarne Thal (446558) and Marius Wrobel (456734)

RWTH Aachen University, Germany
{bjarne.thal, marius.wrobel}@rwth-aachen.de

1 Introduction

Over the past decade, deep neural networks have become the dominant approach for a broad range of perception tasks, from image classification and speech recognition to natural language understanding; in many cases matching or surpassing human performance [lecun2015deep; he2015delving]. Due to their strength, such models are increasingly deployed in critical settings, such as autonomous driving [grigorescu2020survey] or biometric identification [schroff2015facenet].

This success, however, is built upon a surprisingly fragile foundation. In 2013, Szegedy et al. [szegedy2013intriguing] showed that state-of-the-art classifiers can be fooled by *adversarial examples*: inputs altered by a perturbation so small that it is imperceptible to a human, yet enough to make the model produce a confident but incorrect prediction. Such examples are neither rare accidents nor isolated mistakes of a single model: the same perturbed input frequently fools different architectures trained on different data, furthermore the vulnerability appears across all kinds of machine learners [goodfellow2014explaining; biggio2018wild].

This report serves as a broad introduction to the topic of adversarial attacks. In our examination of specific attack methods, we thus concentrate on the classical setting of evasion and poisoning attacks against image classifiers, surveying the ideas and methods that have shaped the research area.

The rest of the report is organised as follows. Section 2 defines adversarial examples, introduces a taxonomy of adversarial attacks, and reviews the most influential evasion and poisoning attacks. Section 3 then contains our discussion: the pertinence of adversarial attacks in the real and physical world, the competing attempts to explain why adversarial examples exist in the first place, and a reflection on what makes the phenomenon a worthwhile object of study.

2 Adversarial Attacks

2.1 Definition

Adversarial attacks can be described as inputting *adversarial examples* into a machine learning system [brendel2017decision]. These two terms were introduced by Szegedy et al. [szegedy2013intriguing] and have since been used in different variations; here, we use the following definition:

Definition 1 (Adversarial Example). *An adversarial example is an input to a machine learning model that has been intentionally perturbed, typically in a subtle and often imperceptible manner, such that the model produces an incorrect prediction while the semantic content of the input remains unchanged.*

Machine Learning Models. While adversarial examples are most commonly studied in the context of deep neural network classifiers [szegedy2013intriguing; madry2017towards; pitropakis2019taxonomy], the phenomenon is not limited to neural networks. Adversarial vulnerabilities have been demonstrated across a wide range of machine learning architectures, including decision trees, support vector machines, k -nearest neighbour classifiers, logistic regression models, ensemble methods and more. This suggests that susceptibility to adversarial manipulation is a property of machine learners in general rather than a characteristic of a specific machine learning method [papernot2016distillation; szegedy2013intriguing; goodfellow2014explaining].

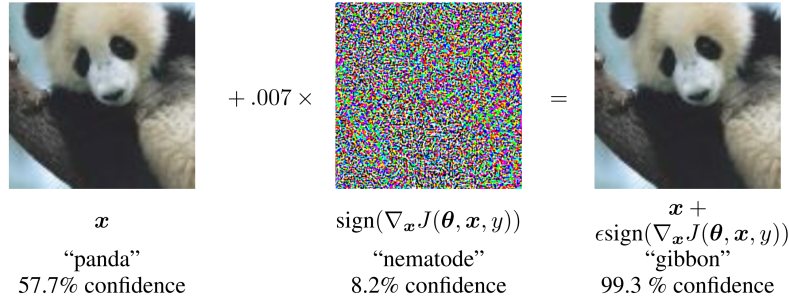


Figure 1: Classic illustration of an adversarial example. A small perturbation that is nearly imperceptible to humans is added to an image of a panda. While the original image is correctly and confidently classified as a panda, the perturbed image is misclassified as a gibbon; also with high confidence. Adapted from [goodfellow2014explaining].

Intentional Perturbations. The requirement that perturbations are *intentional* reflects the modern understanding of adversarial examples. The seminal work of Szegedy et al. [szegedy2013intriguing] demonstrated the existence of inputs that caused misclassification despite appearing unchanged to humans, but did not initially frame these inputs primarily as the result of a deliberate attack. As the field evolved, researchers showed that such inputs can be generated systematically through optimisation procedures and exploited by an adversary [moosavi2016deepfool; madry2017towards]. Consequently, adversarial examples are now typically discussed in the context of *adversarial attacks*, where perturbations are deliberately crafted to elicit faulty model behaviour [chakraborty2018adversarial; pitropakis2019taxonomy; costa2024deep].

Subtle and Imperceptible Perturbations. The notion of *subtlety* for such perturbations can be interpreted in two related but distinct ways. First, adversarial perturbations are commonly constrained to be mathematically small according to a chosen distance metric. These norms define the *perturbation budget* available to an adversary and provide a mathematical notion of similarity between inputs. We will discuss these different norms in more detail in Section 2.2.

Second, adversarial examples are often understood to be imperceptible to humans. In the image domain, this means that the perturbed input should be visually indistinguishable or at least hardly distinguishable from the original image, preserving its semantic meaning [madry2017towards; moosavi2016deepfool]. Although we rarely see the perceptibility of the applied perturbations rigorously validated it is nevertheless very common for authors to state, based on a visual assessment, that the adversarial examples they have generated are indistinguishable from the original to humans [brendel2017decision; chakraborty2018adversarial; costa2024deep; liang2022adversarial; papernot2016distillation; papernot2016transferability].

2.2 Taxonomy

Adversarial attacks can be categorised according to several orthogonal dimensions which we are presenting in this section. These dimensions are widely used throughout the literature and provide a useful framework for comparing attack methodologies [pitropakis2019taxonomy; costa2024deep]. The attacks discussed in this survey are classified according to this taxonomy, with a complete overview provided in Table 1.

By Attacker Knowledge One of the most common distinctions is based on the amount of information available to the attacker about the target model [pitropakis2019taxonomy; costa2024deep].

White-box Attacks. In a *white-box* setting, the attacker has complete knowledge of the target model, including its architecture, parameters, training procedure, and gradients. This level of knowledge makes highly effective optimisation-based attacks possible that directly exploit gradient information to identify adversarial perturbations. Many influential attacks, including the Fast Gradient Sign

Method (FGSM), Projected Gradient Descent (PGD), and the Carlini-Wagner attack, were originally proposed in this setting. White-box attacks are commonly used to evaluate the worst-case robustness of machine learning models.

Black-box Attacks. In a *black-box* setting, the attacker has no knowledge about the internal structure of the model and can only observe its outputs in response to supplied inputs. Such attacks more closely resemble many real-world scenarios, where proprietary models are exposed only through prediction interfaces or APIs.

By Attack Stage Attacks can further be classified according to the stage of the machine learning pipeline at which they are carried out [chakraborty2018adversarial; pitropakis2019taxonomy].

Evasion Attacks. *Evasion attacks* are performed during the inference phase after a model has been trained. The attacker modifies input samples in order to induce misclassification while avoiding detection. Since such attacks they require no influence over the training process, they are considered the most practical and widely studied form of adversarial attack.

Poisoning Attacks. *Poisoning attacks* target the training phase by injecting maliciously crafted samples into the training dataset. The objective here can be degrading the model's overall performance, manipulating its behaviour for specific inputs, or implant hidden backdoors that can later be triggered by an attacker. While poisoning attacks require some influence over the data collection or training pipeline, this assumption is realistic in settings that rely on crowdsourced, user-generated, or automatically collected data.

By Attack Goal The third dimension considers the attacker's intended outcome [chakraborty2018adversarial; costa2024deep; brendel2017decision].

Targeted Attacks. *Targeted attacks* seek to persuade the model to produce a specific attacker-chosen output. Drawing on the example given in 1, an attacker might attempt to modify an image of a panda such that it is classified as a gibbon. Because the attack must drive the prediction toward a particular target class, targeted attacks are generally more difficult to perform than untargeted attacks.

Untargeted Attacks. *Untargeted attacks* aim only to cause a prediction error, but do not specify the resulting incorrect class. The attack is therefore successful as soon as the model's prediction differs from the correct label. Again using the example from 1, the attacker might aim at having the classifier recognize the perturbed image as showing anything but a panda.

By Perturbation Constraint Adversarial attacks are commonly categorised according to the distance metric used to constrain the perturbation applied to the input [costa2024deep; liang2022adversarial]. These constraints define what constitutes a *small* modification and therefore determine the attacker's search space [carlini2017towards]. The majority of attacks in the literature operate under one of three norm-based threat models. Figure 2 illustrates these three norms geometrically.

L_0 -Constrained Attacks. The L_0 norm measures the number of features that have been modified. In the context of image classification, this corresponds to the number of altered pixels regardless of the magnitude of the changes. An example is the L_0 variant of the Carlini-Wagner attack [carlini2017towards].

L_2 -Constrained Attacks. The L_2 norm measures the Euclidean distance between the original and perturbed input vectors. Under this constraint, perturbations may be distributed across many features as long as the overall magnitude remains small [liang2022adversarial]. Due to its intuitive geometric interpretation, the L_2 norm is widely used when evaluating adversarial robustness [costa2024deep].

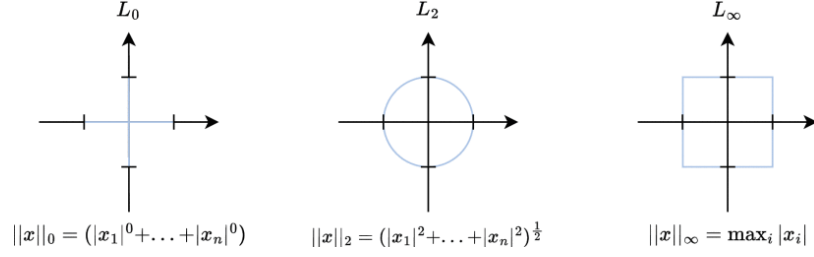


Figure 2: Geometric interpretation of the three norm constraints commonly used in adversarial machine learning. The unit balls of the L_0 , L_2 , and L_∞ norms are shown in a two-dimensional feature space. The L_0 norm constrains the number of modified features, the L_2 norm constrains the Euclidean distance between the original and perturbed input, and the L_∞ norm constrains the maximum change applied to any individual feature. Figure adapted from [costa2024deep].

Table 1: Overview of the attacks discussed in this survey, classified along the four dimensions of the taxonomy in Section 2.2. “Both” marks attacks with widely used targeted and untargeted variants; “-” marks poisoning attacks for which an input-space norm budget does not directly apply.

Attack	Knowledge	Stage	Goal	Constraint
L-BFGS [szegedy2013intriguing]	White-box	Evasion	Targeted	L_2
FGSM [goodfellow2014explaining]	White-box	Evasion	Untargeted	L_∞
BIM / iterative [kurakin2017physical]	White-box	Evasion	Both	L_∞
PGD [madry2017towards]	White-box	Evasion	Untargeted	L_∞
DeepFool [moosavi2016deepfool]	White-box	Evasion	Untargeted	L_2
Carlini-Wagner [carlini2017towards]	White-box	Evasion	Targeted	$L_0/L_2/L_\infty$
ZOO [chen2017zoo]	Black-box	Evasion	Both	L_2
BadNets [gu2017badnets]	White-box	Poisoning	Targeted	-
Targeted backdoor [chen2017targeted]	Black-box	Poisoning	Targeted	-
Poison Frogs [shafahi2018poison]	White-box	Poisoning	Targeted	-
Clean-label backdoor [turner2019label; saha2020hidden]	White-box	Poisoning	Targeted	-
Narcissus [zeng2023narcissus]	Black-box	Poisoning	Targeted	L_∞

L_∞ -Constrained Attacks. The L_∞ norm limits the maximum change that can be applied to any individual feature [costa2024deep; liang2022adversarial]. Many influential attacks, including FGSM and PGD, operate under an L_∞ constraint because it provides a straightforward approximation of perceptual similarity while enabling efficient optimisation [goodfellow2014explaining; madry2017towards].

2.3 Notable Evasion Attacks

This section presents several notable adversarial attacks that have significantly influenced the development of the research area. An overview of these attacks, together with their classification according to the taxonomy introduced in Section 2.2, is provided in Table 1. Throughout the following sections we follow the notation of the respective original papers; the same symbols (e.g. δ , ϵ , η , f) may therefore denote different quantities in different subsections.

L-BFGS The first widely recognised adversarial attack was introduced by Szegedy et al. [szegedy2013intriguing]. Their work did not only show that machine learning models are vulnerable to inputs that lead to incorrect predictions while appearing unchanged to human observers, first describing the phenomenon of an adversarial example. They furthermore introduced a first reliable method to find such examples.

To generate adversarial examples, the authors formulated the problem as a constrained optimisation task. Given an input x and a target class l , the objective is to find the smallest perturbation r such that the perturbed input $x + r$ is classified as the target class:

$$\min_r \|r\|_2 \quad \text{subject to} \quad f(x+r) = l, \quad (1)$$

where f denotes the classifier and $\|r\|_2$ represents the Euclidean distance between the original and perturbed input. Directly solving this problem is difficult since the classification constraint $f(x+r) = l$ is highly non-linear and non-convex. To make the problem handleable, Szegedy et al. reformulated it as a continuous optimisation problem by replacing the hard classification constraint with a differentiable loss term that encourages the model to predict the target class. Specifically, they minimise

$$\min_r c\|r\|_2 + \text{loss}_f(x+r, l), \quad (2)$$

subject to the box constraint

$$x+r \in [0, 1]^m, \quad (3)$$

where $\text{loss}_f(x+r, l)$ measures how strongly the network predicts class l . The parameter $c > 0$ controls the trade-off between achieving the target misclassification and keeping the perturbation small, and the box constraint ensures that the perturbed input remains a valid image. This reformulation yields a differentiable objective whose gradients can be computed via backpropagation, allowing the perturbation to be found efficiently using the box-constrained Limited-memory Broyden-Fletcher-Goldfarb-Shanno (L-BFGS) optimisation algorithm. A line search over c is then performed to identify the smallest successful perturbation that causes the classifier to output the target class.

Although the L-BFGS attack is still computationally expensive compared to later methods, its significance lies in motivating the extensive research that followed. For this reason, it is widely regarded as the starting point of adversarial machine learning research [goodfellow2014explaining; costa2024deep; liang2022adversarial].

Fast Gradient Sign Method (FGSM) Goodfellow et al. hypothesized that the reason adversarial examples exist is that commonly, models are designed to behave mostly linear [goodfellow2014explaining]. Previously, it was suspected that models were too non-linear [szegedy2013intriguing].

Based on this hypothesis, they came up with the fast gradient sign method to find L_∞ -constrained adversarial examples. L_∞ is chosen because it models sensor inaccuracies: a small perturbation in a sensor value would still be encoded to the same digital value. Unlike L-BFGS, they did not choose a specific target class they want the adversarial example to be mistaken for. Instead, they aim to maximise the loss value of the adversarial example. That means that we want to solve

$$\arg \max_{x' \text{ s.t. } \|x'-x\|_\infty \leq \delta} L(\theta, x', y) = x + \arg \max_{\|\eta\|_\infty \leq \delta} L(\theta, x + \eta, y)$$

given a data point x , a limit δ , and a loss function $L(\theta, x, y)$, to find the ‘worst’ adversarial example (maximises loss) in the L_∞ -ball around x

To do that, Goodfellow et al. linearise the loss function around x , giving us

$$\arg \max_{\|\eta\|_\infty \leq \delta} L(\theta, x + \eta, y) = \arg \max_{\|\eta\|_\infty \leq \delta} L(\theta, x, y) + \nabla_x L(\theta, x, y)^T \eta = \arg \max_{\|\eta\|_\infty \leq \delta} \nabla_x L(\theta, x, y)^T \eta$$

We can easily see that $\eta = \delta \cdot \text{sign}(\nabla_x L(\theta, x, y))$ is a valid maximiser of the scalar product $\nabla_x L(\theta, x, y)^T \eta$. This corresponds to a single, as large as possible step in the direction of the gradient.

Since this approach only needs one backpropagation step instead of iterations like an L-BFGS algorithm run, this was called the Fast Gradient Sign Method. It made adversarial training feasible, which aims to improve a model’s robustness by inserting adversarial examples into the training set.

Variants of FGSM Different variants of FGSM were proposed by Kurakin et al. [kurakin2017physical; kurakin2017scale].

The so-called **basic iterative method** applies FGSM multiple times with a small step size [kurakin2017physical]. The adversarial example is given by

$$X_0^{adv} = X, \quad X_{N+1}^{adv} = \text{Clip}_{X,\epsilon}(X_N^{adv} + \alpha \text{sign}(\nabla_X L(X_N^{adv}, y_{true})))$$

where $\text{Clip}_{X,\epsilon}$ implements pixel-wise constriction to the ϵ -neighborhood of X [kurakin2017physical].

To find adversarial examples of a specific class y_{target} instead of maximising the loss function, FGSM can be modified to maximise that classes probability $p(y_{target}|X)$. This can be achieved with a single step in direction of $\text{sign}(\nabla_X \log(p(y_{target}|X)))$ [kurakin2017physical], analogous to basic FGSM, or using multiple, like in the aforementioned basic iterative method [kurakin2017scale]. A popular choice for target class is the one originally deemed least likely by the target model.

PGD Searching adversarial examples by using projected gradient descent (PGD) as proposed by Madry et al. [madry2017towards] is similar to the basic iterative method. Both use first-order information, i.e. the gradient of the loss function, to iteratively maximise the loss on an input in the neighborhood of the real image. Instead of using multiple FGSM steps, though, Madry et al. use projected gradient descent directly.

They formulate the adversary's goal as

$$\max_{\eta \in S} L(\theta, x + \eta, y)$$

Note the similarities to FGSM, by using $S = \{\eta \mid \|\eta\| \leq \delta\}$, we obtain its maximization problem. FGSM is therefore a specific method to tackle this general formulation.

As a generalised, stronger attack, they propose using PGD. They support their argument with an experiment, starting PGD from random points in the vicinity of x . While many different local maxima are found, they have roughly the same loss value. If an even better maximum existed, they argue, it would be hard to find with any first-order method. Therefore, no other first-order method could find it in reasonable time either, making PGD a useful "universal" first-order adversary.

However, this strength has a caveat. As we will see in Section 2.3, not every adversarial example corresponding to a local minimum is equally transferable to other models.

DeepFool DeepFool was introduced by Moosavi-Dezfooli et al. [moosavi2016deepfool] in 2016. The central idea of this iterative method is to characterise adversarial vulnerability geometrically: rather than loosely optimising an attack loss, DeepFool estimates the distance of an input to the closest decision boundary. For affine classifiers, this distance could be calculated in a closed form by projecting the input point onto the decision hyperplane. The highly nonlinear decision boundaries of deep neural networks are addressed by DeepFool by exploiting the observation that although the true boundary of a neural network may be highly curved, sufficiently small neighborhoods can often be approximated accurately by a hyperplane. The shortest path to this approximate boundary is assumed to provide a good estimate of the minimal adversarial perturbation. This auxiliary hyperplane is recomputed at each iteration through a first-order Taylor expansion of the classifier around the current point; as the point is repeatedly projected onto the latest hyperplane, it moves closer and closer to the true, nonlinear decision boundary until it eventually crosses it. In mathematical terms, at an intermediate point x_i , the classifier is iteratively displaced by its local linear approximation,

$$f(x) \approx f(x_i) + \nabla f(x_i)^T (x - x_i), \quad (4)$$

which defines a locally linear decision boundary. The perturbation is obtained by projecting the current point onto this locally estimated boundary, yielding an update

$$x_{i+1} = x_i + r_i, \quad (5)$$

where r_i is the smallest perturbation that crosses the linearised boundary. The procedure is repeated until the classifier changes its prediction.

For multiclass classifiers, DeepFool extends this idea by considering the set of pairwise decision boundaries between the current class and competing classes. At each iteration, the method computes the locally closest boundary under the linear approximation and updates the input toward that boundary. Figure 3 visualises the iteration step.

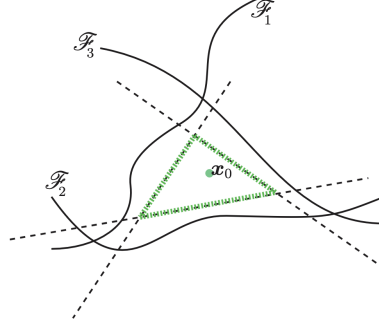


Figure 3: The approach of DeepFool estimates the position of the decision boundaries with the other classes with a linear representation using first-order Taylor expansion. By iteratively projecting the input point onto this auxiliary hyperplane, the perturbed point moves closer and closer to the actual decision boundary of the model until eventually crossing it.

Maybe the most important contribution of DeepFool to the field of adversarial attacks is the geometrical perspective that it shed on the problem: instead of treating adversarial generation as a generic constrained optimisation problem, it directly exploits the local structure of the classifier and latches onto a geometrical image of the decision boundary that needs to be crossed. Furthermore, it gave the L-BFGS attack of Szegedy et al. [szegedy2013intriguing] a new, which had required solving a computationally expensive optimisation problem for each input. By using gradient information to directly estimate the boundary geometry, DeepFool achieved smaller perturbations than earlier methods while requiring considerably less computation than general-purpose optimisation approaches.

Carlini-Wagner The Carlini-Wagner (C&W) attack is an influential attack presented by Carlini and Wagner [carlini2017towards]. The method builds directly upon the original L-BFGS method by Szegedy et al. [szegedy2013intriguing] by substantially improving the objective function of the optimisation problem. They further introduced a trick to formulate the problem as an unconstrained optimisation task, instead of a constrained one.

Just like L-BFGS, given an input example x and a target class t , the attack seeks an adversarial example $x' = x + \delta$ by solving:

$$\min_{\delta} \|\delta\|_p + c \cdot f(x + \delta), \quad (6)$$

where $\|\delta\|_p$ measures the size of the perturbation and f is a logit-based loss function that operates on the network's raw logits rather than the softmax output probabilities. The loss is defined as:

$$f(x') = \max \left(\max_{i \neq t} Z_i(x') - Z_t(x'), -k \right), \quad (7)$$

where $Z_i(x')$ denotes the logit corresponding to class i , and k is a confidence parameter controlling the margin by which the target class must exceed competing classes. This formulation directly optimises against the decision boundary of the classifier and thus efficiently calculates adversarial examples.

To address the pixel-range constraint of image inputs, the C&W attack introduces a clever variable transformation:

$$x' = \frac{1}{2}(\tanh(w) + 1), \quad (8)$$

where w is an unconstrained latent space optimisation variable. This transformation converts the original constrained optimisation problem into an unconstrained one while guaranteeing that the generated adversarial example remains within the valid input range. The attack then optimises w using gradient-based optimisation methods, typically the Adam optimiser [kingma2014adam].

The original work introduced variants based on different distance metrics, including L_0 , L_2 , and L_∞ norms, allowing the attack to evaluate robustness under different threat models.

The effectiveness of the C&W attack also demonstrated weaknesses in several proposed defenses. In particular, it was shown to successfully bypass defensive distillation [papernot2016distillation], a technique proposed to improve adversarial robustness by training a model using softened output probabilities from a teacher network. Although defensive distillation appeared to increase resistance against earlier attacks, Carlini and Wagner demonstrated that the method did in fact only obscure the model's gradient albeit not truly hiding it. By directly targeting the logits with their loss function, Carlini and Wagner were able to efficiently generate adversarial examples in spite of defensive distillation by directly approximating the obscured decision boundary.

Transfer Attacks In a transfer attack, an adversary first obtains a substitute model that he has full access to as a stand-in for a target black-box model. On this substitute, white-box attacks can be performed to find adversarial examples. Since they generally transfer from one model to another, these can then be used to attack the actual target model.

Jacobian-Based Dataset Augmentation Papernot et al. [papernot2017practical] introduced black-box attacks via transfer attacks where neither target model parameters nor a suitable data set is accessible to the attacker. Prior work by Biggio et al. [biggio2013evasion] had introduced transfer attacks already, but granted access to such a dataset.

First, the substitute architecture is defined based on input and output of the target, e.g. one adapted to images. The architecture is overall not a deciding factor in the success of this approach.

The dataset restriction poses a difficulty for training the substitute. Papernot et al. solve this by querying the target on artificial data points to create a substitute dataset. Here, the selection of good data points to learn similar decision boundaries is crucial: a bad crafting strategy would require infeasible and highly suspicious amounts of queries to the target model. In particular, generation based on random noise does not work.

Instead, they rely on a heuristic to find out in which input direction the (substitute!) model's output changes and adding new samples focused on these directions. For that, they rely on the substitute model's Jacobian matrix (multidimensional derivative, each entry is one output coordinate partially derived by an input coordinate) at input points.

They start by collecting a set of samples somewhat representative of the target's input domain, such as different natural images. Next, an architecture is selected.

The following steps are then performed repeatedly: The adversary makes the target model label his current selection of samples, and then trains his substitute with the new examples. Then, he augments his dataset with additional input points.

Let $J_f(x)$ be the Jacobian of the substitute at a point x . Then, $J_f(x)[o(x)]$ is the column corresponding to partial derivatives of the correct label (i.e. the one assigned by the target model). Papernot et al. now add the sign of this column scaled by a factor λ to point x : $x = x + \lambda \text{sign}(J_f(x)[o(x)])$, and add it to their dataset.

By doing this to all current points, they obtain a larger sample set, so the model trained in the next iteration is closer to the target. Note that every epoch, the model is trained from scratch. With enough iterations, the substitute can then be used to generate the adversarial examples.

Ensemble Attacks To improve transferability of adversarial examples created on substitute models, Liu et al. [liu2016delving] argue that an adversarial example that works well for multiple models at the same time is bound to transfer better than one that only works on a single one.

They therefore propose that an adversary in the transfer attack setting crafts adversarial examples against an ensemble of models. For a targeted attack, they suggest the negative-log-likelihood-objective:

$$x^* = \arg \min_{x'} -\log((\sum_{i=1}^k \alpha_i J_i(x')) * \mathbb{1}_{y^*}) + \lambda \|x - x'\|$$

where $J(*)$ is each models output, α_i scaling factors for each such that $\sum_i \alpha_i = 1$, for example simply $1/k$.

MI-FGSM and Momentum Kurakin et al. [kurakin2016adversarial] remarked that, while iterative methods yield better results on a single attacked white-box model, single-step methods exhibit better transferability. They suspected that iterative methods overfit to parameters of the model.

To tackle this issue, Dong et al. [dong2018boosting] proposed adding momentum to iterative methods such as iterative FGSM, leading to momentum iterative FGSM. This way, both the underfitting of basic FGSM as well as the overfitting of iterative FGSM is eased.

The gradient is updated as

$$g_{t+1} = \mu * g_t + \frac{\nabla_x J(x_t^*, y)}{\|\nabla_x J(x_t^*, y)\|_1}$$

with μ being the decay factor and the gradient being normalised to handle gradient magnitudes varying over iterations.

Accordingly, the sample is updated as

$$x_{t+1}^* = x_t^* + \alpha * \text{sign}(g_{t+1})$$

α being the step size. For $\mu = 0$, this goes back to being normal FGSM. Note that this change is not restricted to FGSM, but can be applied to any iterative gradient-based method including targeted ones.

ZOO In 2017, Chen et al. [chen2017zoo] proposed Zeroth-Order Black Box Attacks (ZOO), a black-box attack without the use of a substitute model. This way, they avoided losses in transferability to the target and achieved comparable results to then-state of the art white box attacks.

This attack is possible if the adversary is allowed to query the target model, i.e. obtaining the confidence scores, $f(x)$ for an arbitrary x . This might be the case for a classification-as-a-service tool. Gradients and model configuration, however, are not available (black box).

To circumvent this restriction, Chen et al. estimate a pseudo-gradient using two queries per coordinate:

$$\hat{g}_i := \frac{\partial f(x)}{\partial x_i} \approx \frac{f(x + he_i) - f(x - he_i)}{2h}$$

where f is a loss function defined on the model's output and h is set to a small constant instead of approaching 0 like in the derivation formula. Naively, this would suffice to estimate full gradients and perform gradient descent. However, this formula requires two evaluations per coordinate, $f(x + he_i)$ and $f(x - he_i)$. Since each pixel translates to three coordinates of input data, this is not a feasible approach.

Instead, Chen et al. perform stochastic coordinate descent, improving just a batch of coordinates at each step. This way, they do not need to estimate the full gradient each time, similar to how stochastic gradient descent only uses a batch of samples at each step. For the optimisation itself, they use ADAM or Newton's method.

Still, we need to find a suitable loss function f . In fact, we can use different loss functions to adapt different white-box attacks. In their work, Chen et al. adapted the Carlini-Wagner attack. C-W's loss function performs on the raw logits of a model, an information inaccessible in the black-box case.

Instead, Chen et al. use the logarithm of the confidence scores as a proxy of sorts for the logits and obtain

$$f(x, t) = \max_{i \neq t} \{\max\{\log[F(x)]_i\} - \log[F(x)]_t, -\kappa\}$$

for targeted attacks and

$$f(x, t) = \max\{\log[F(x)]_{t_0} - \max_{i \neq t_0}\{\log[F(x)]_i\}, -\kappa\}$$

for untargeted ones.

2.4 Notable Poisoning Attacks

Poisoning attacks were already included in the early taxonomy of Barreno et al. [barreno2010security], which clustered attacks on machine learning into exploratory and causative, analogous to our evasion and poisoning taxonomy.

The first poisoning attack that got greater attention was Biggio et al.'s gradient-based attack on Support Vector Machines [biggio2012poisoning]. Subsequently, more research was contributed, especially on attacks on neural networks.

Backdoor Poisoning Attacks With backdoor attacks on machine learning as proposed by Chen et al. [chen2017targeted] and Gu et al. [gu2017badnets], the adversary's goal is to introduce a backdoor for himself into the model. While the performance on normal inputs is to be kept up, the model should react to specific inputs by misclassifying them. (Either generally, or targeted.) These specific inputs carry a special pattern, the "key" or "trigger".

Gu et al. consider an adversary that has full access to the model or a base version of it. Namely, an ML-as-a-service provider (full access) and a provider of pretrained models (base version access). In both cases, the architecture is specified already, as well as a training set. A backdoor key pattern is chosen by the attacker. The adversary has to find model weights performing well on general samples from the set yet vulnerable to the backdoor.

For that, Gu et al. use training set poisoning, adding a fraction of training images to the dataset again, only with the trigger pattern included and the labels changed according to varying strategies. They did not yet derive a way to optimise poison samples, instead, pattern, label change formula and infected fraction need to be selected by the adversary.

Chen et al. instead use a black-box threat model where the adversary does not have access to model architecture or training data and can only inject a small number of adversary samples including arbitrary labels. The key should not be easily noticeable by a human observer. Crucially, the model performance on non-backdoor inputs should be kept up.

Backdoor keys may either be a single instance or a pattern added to instances. The former ("input-instance-key" strategy) allows injecting just very few poisoned examples while the latter ("pattern-key") can be used to craft adversarial examples from a wide range of inputs, which is often more useful to a hypothetical attack.

In input-instance-key strategies, Chen et al. propose adding different noised versions of the key-to-be to the training set, each with the chosen label. The idea is to get the model to overfit to these.

In Pattern-key strategies, the adversary can blend a key pattern onto pictures, add an accessory, or blend in an accessory according to Chen et al.'s suggestions. The idea of the blending is to make the pattern harder to spot for humans. In any case, the perturbed sample is added to the dataset as poison.

Like Gu et al.'s attack, theirs also depends on hyperparameter choices only, with the framework not optimising poisoning on its own.

Clean-Label Attacks Notably, while the backdoor poisoning attack described by Chen et al. does not need access to the model, the adversary has to be able to assign arbitrary labels to samples, as does the one by Gu et al. However, this can be highly noticeable and alert human overseers to the attack or might just not be feasible in another way. Clean-label attacks do not give such power to the adversary.

One such attack was proposed by Shafahi et al. [shafahi2018poison]. In contrast to Chen et al.'s attack, the adversary does know the model parameters, which might happen if the target model

is based on a pretrained, freely accessible one. His goal is to have a target instance misclassified without modifying it, e.g. getting a malware script to bypass a ML antivirus.

First, he chooses a base instance, such that its class, the "base class", is the one he wants the target instance to be classified as. With this, he crafts a poison instance p by solving

$$p = \arg \min_x ||f(x) - f(t)||_2^2 + \beta ||x - b||_2^2$$

where the first term compares the target t and poison candidates x in feature space (the more similar, the better), and the second penalises difference from the base instance. The general idea is to find a poison instance close to b , therefore likely to be labeled the same, yet close to t in feature space (output of second-to-last layer), so t is treated the same as b by the model. For optimisation, two steps are iterated: First, a gradient step ("forward step"), then a step towards b ("backwards step").

Clean-Label Backdoor Attacks Saha et al. [saha2020hidden] made an effort to combine clean-label attacks with backdoor poisoning attacks, albeit Turner et al. did the same a year earlier [turner2019label]. Both also considered how to hide the trigger during poisoning to decrease the noticeability of the backdoor, though differently. Turner et al. manually select a "trigger amplitude", essentially an opaqueness. Saha et al., on the other hand, directly incorporate it into their optimisation and not even have the pattern itself in the poisoned instance.

Saha et al. consider a threat model in which the adversary can add poison to a dataset the victim uses for fine-tuning a pretrained model, which the adversary has access to. The scenario is similar to the one described by Gu et al., with the difference that the adversary attacks the fine-tuning, not the pretraining.

The attack itself is similar to Shafahi et al.'s. The adversary chooses a source and a target image from the data distribution. He then applies his trigger to the source, obtaining the patched source. Like before, the poison sample is optimised to be close to the target in pixel space (so it probably receives the target label) while close to the patched source in feature space (so the model has difficulties distinguishing them). Note that the roles of source and target are flipped in the notation compared to Shafahi et al.

Let $\tilde{s}$ be the patched source, t the target, and ϵ a small, chosen threshold. Saha et al. optimise

$$\arg \min_x ||f(x) - f(\tilde{s})||_2^2 \quad \text{s.t.} \quad ||f(x) - f(t)||_\infty^2 < \epsilon$$

to obtain the poisoned sample x . Again, f is the output of the second-to-last layer, the feature space representation. To optimise, they use projected gradient descent (Section 2.3). However, in each optimisation step, a new randomly-patched source is selected so the backdoor generalises to arbitrary images and patch placements.

A more recent work by Zeng et al. [zeng2023narcissus] addresses issues with prior work on clean-label backdoor attacks. They consider a black-box threat model where the adversary can only poison a small subset of the training data. The adversary chooses the target class and knows some examples in it. He also has general information on the task the victim model should perform, such as "face recognition", but not specifics like classes used. This enables the attacker to draw up more data instances related to the task, though they might not belong to the original data distribution and are therefore referred to as "public out of distribution (POOD) examples". This is especially easy in image or language tasks where lots of samples are available in the web.

All in all, the adversary is considerably weaker than in aforementioned attacks.

Their approach revolves around optimising the trigger pattern used under a constraint so it is harder to notice for humans (as in evasion attacks). Suppose the attacker had access to the model the victim would obtain without poisoning. We call this model the oracle f_{orc} . Recall that he also has access to a dataset of the target class D_t . Zeng et al.'s approach is to choose a trigger δ^* maximising the oracle's confidence on the target class, i.e. solving

$$\delta^* = \arg \min_{\delta \in \Delta} \sum_{(x,t) \in D_t} L(f_{orc}(x + \delta), t)$$

where Δ is the set of allowed perturbations, e.g. the L_∞ -ball. This way, δ^* itself is related to the target class, and adding it to any input increases the likelihood of that input being classified as the target class.

However, the adversary does not have access to f_{orc} . Instead, Zeng et al. propose training a surrogate model f_{sur} on the POOD data D_{POOD} . The above equation becomes

$$\delta^* = \arg \min_{\delta \in \Delta} \sum_{(x,t) \in D_t} L(f_{sur}(x + \delta), t)$$

Now, the attack works as follows. First, f_{sur} is trained using D_{POOD} and D_t . While training the surrogate model on both D_{POOD} and D_t simultaneously would be possible, Zeng et al. suggest training on D_{POOD} first and fine-tune on D_t to make modification for a possible second attack on another class more efficient.

In the second step, they use (stochastic) projected gradient descent to find the trigger δ^* and subsequently add it to a portion of the target class samples D_t . The victim is then given this poisoned target class data set.

In an eventual exploit, the adversary adds the trigger to the desired input. It is an option to amplify the trigger to increase attack success rates.

3 Discussion

3.1 Real-World and Physical Adversarial Attacks

Somewhat surprisingly, in practice, confirmed *malicious* adversarial-example attacks against real institutions or individuals remain strikingly rare [biggio2018wild; raff2023dont]. Two explanations seem possible.

1. Adversarial attacks slip through unnoticed: a successful adversarial input may be logged as an ordinary misclassification, be considered a generic model error, or filed under an unrelated incident category, so that successfully employed adversarial attacks fly under the radar.
2. Adversarial attacks might in practice simply be of little use to actual attackers. Crafting them reliably demands specialised machine-learning and security expertise, whereas cruder techniques—distributed denial-of-service, character obfuscation to slip past spam filters, or merely packing a malware binary—remain cheap, fast, and effective [apruzzese2023real; raff2023dont].

The second explanation is the one that we consider to be more plausible. Building an adversarial example to defeat a system that a simpler attack would already defeat is like learning to pick a lock when the crowbar is cheaper, quicker, and works just as well. Interestingly, the few confirmed real-world cases, such as commercial CAPTCHA-solving services or the prompt injection of deployed chatbots, tend to be those where no such simpler alternative exists.

This does not render the phenomenon academic in the pejorative sense. As machine learning is embedded ever more deeply into the physical world—autonomous vehicles, automated border control, biometric surveillance—the question of whether an input that is semantically unchanged to a human can nonetheless fool a sensor pipeline becomes a central engineering concern. The body of work on *physical* adversarial attacks shows that it can: inconspicuous stickers on a stop sign reliably cause targeted misclassification, both in the laboratory and from a moving vehicle; [eykholt2018robust] printable adversarial patches force a chosen label onto almost any scene they appear in; [brown2017adversarial] adversarial eyeglass frames defeat face recognition; [sharif2016accessorize] and spoofed sensor returns or crafted road-surface markings can mislead the perception stack of a production self-driving system [cao2019adversarial; tencent2019tesla].

Such results matter because robustness is a core requirement for any safety-relevant deployment—which makes it all the more pressing to understand *why* such systems fail, and what that failure implies. It is to those questions that we turn in the following section.

3.2 Explaining Adversarial Examples

Having seen *where* adversarial examples arise, we now turn to *why* they exist, highlighting some influential explanation attempts that have been proposed since their discovery in 2013.

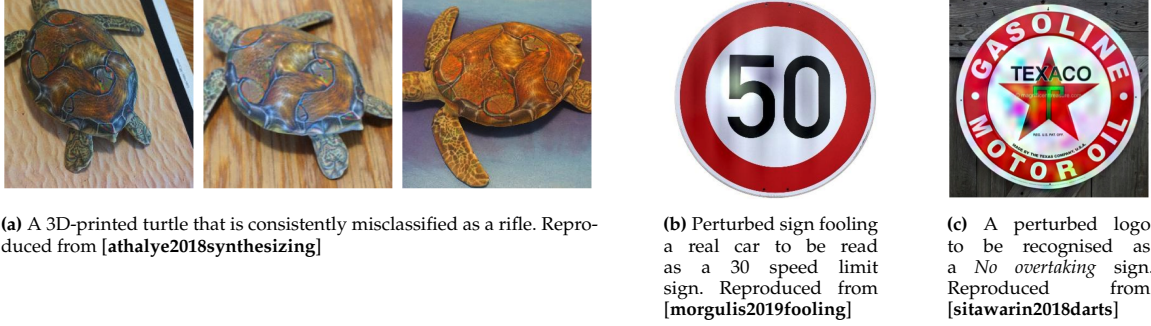


Figure 4: Many creative approaches for transferring adversarial examples into the physical world have been proposed in academic writing, three of which we are showing here. As machine learning applications experience increasingly widespread adoption in the real world, many researchers warn of points of attack opening up and possibly enabling severe damage.

Mysterious Nonlinearity The initial paper by Szegedy et al. that introduced the concept of adversarial attacks, already made various core observations that shaped research ever since [szegedy2013intriguing]. Firstly, they postulated that adversarial examples are neither random artefacts nor isolated anomalies: they occupy entire regions of the input space—*pockets* in the data manifold—that are sparse, and therefore hard to encounter through random sampling, yet can be produced reliably through optimisation. Second, the effect is not merely overfitting. The same adversarial input often fools different models with different hyperparameters, and even different architectures—the phenomenon later called *transferability*—which already argued against attributing adversarial examples to a single model overfitting its training set.

The missing piece that was not offered by their paper was an explanation of the curious phenomenon. Szegedy et al. do postulate that *highly nonlinear model behaviour* might be behind this issue, but no comprehensive explanation was offered. It was only the next year that such explanation attempt was made.

High-Dimensionality The explanation by Goodfellow et al. [goodfellow2014explaining] can be regarded as one of the most influential in the field. It overturns the intuition, inherited from [szegedy2013intriguing], that adversarial examples stem from the *extreme non-linearity* of deep networks—highly contorted decision surfaces with rare, low-probability blind spots—possibly compounded by overfitting. Goodfellow et al. argued that linear behaviour in high-dimensional spaces is already *sufficient* to produce adversarial examples, refuting the nonlinearity explanation attempt. Because modern networks are often deliberately built to locally behave in near-linear ways (e.g. using ReLU) for ease of optimisation, the argument carries over from strictly linear models to deep networks.

The mechanism is a high-dimensional *bloating* effect. For a linear model with weight vector w , the response to a perturbed input $x + \eta$ —the original input x plus a perturbation η —is $w^\top(x + \eta) = w^\top x + w^\top \eta$, so the perturbation shifts the activation by $w^\top \eta$. The overall output change of a perturbation thus scales linearly with the number of input dimensions n and mean weight magnitude m with respect to the per-pixel change:

$$w^\top \eta = \epsilon \sum_{i=1}^n |w_i| = \epsilon m n.$$

This means that in sufficiently high dimensions, many individually imperceptible, weight-aligned changes accumulate into one large change in the output. Figure 5a illustrates this: the per-pixel budget stays flat as n grows, while the change in the model’s output climbs until it crosses the threshold that flips the prediction.

This point is easily overstated, and often is: Goodfellow et al. do not claim that linearity is a necessary ingredient of the explanation. They show only that non-linearity is not necessary—the simplest possible model, plain logistic or softmax regression with no hidden layer, already exhibits adversarial examples—as they showcased with FGSM which we discussed in section 2.3. (The refined, later

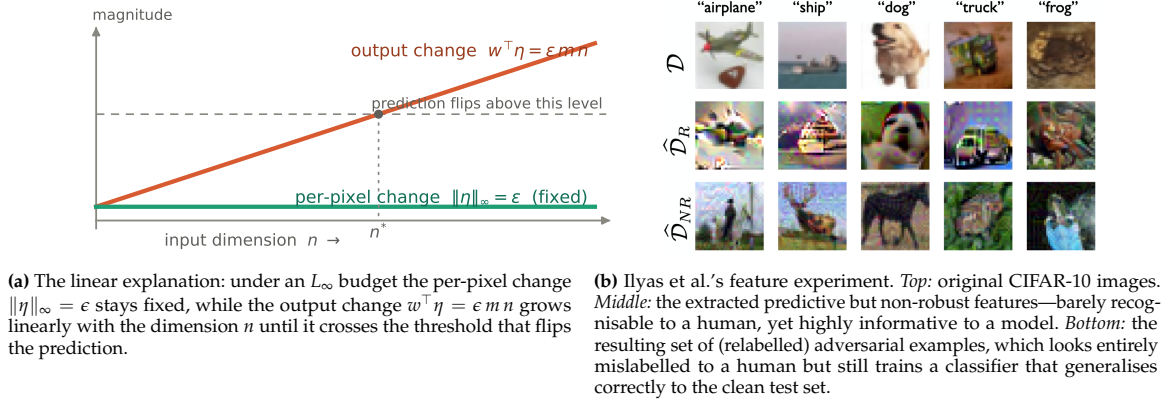


Figure 5: Two complementary explanations for the existence of adversarial examples: (a) high-dimensional linearity [goodfellow2014explaining] and (b) predictive but non-robust features [ilyas2019adversarial].

view holds that both dimensionality and local curvature matter, as reflected in the stronger iterative and minimum-norm attacks discussed above [madry2017towards; moosavi2016deepfool].)

The linear view also provided the first explanation of transferability. Since adversarial examples now occupy broad, contiguous half-spaces rather than fine pockets, different models' adversarial regions overlap heavily, which explains why an example crafted for one model tends to fool another at all. And because the discriminative direction is largely a property of the data rather than of the model, models trained on the same task tend to learn similar weight vectors, so that the same perturbation pushes them toward the same incorrect class—which explains why the models tend to agree on that class.

The Fundamental Mismatch Ilyas et al. [ilyas2019adversarial] shift the framing once more, reframing adversarial examples not as pathological failures but as a *mismatch between machine and human perception*: models exploit patterns that are genuinely predictive, but imperceptible to, and unintuitive for, humans. They distinguish *predictive* features (features correlated with the label) from *robust* features (features that remain predictive under perturbation). Since *empirical risk minimisation* rewards any signal that improves accuracy, a learner will readily rely on predictive but non-robust features; adversarial vulnerability is then an expected consequence of standard training, rooted in the gap between the human sense of similarity and the statistical structure of the data. A striking experiment makes this concrete: they relabel every image of CIFAR-10 with a new, random target class and perturb it so that a model reads the new label, while to a human the images appear unchanged and hence entirely mislabelled. A standard model trained on this dataset nevertheless generalises well to the *clean*, unmodified test set—evidence that it has latched onto real, transferable features that merely happen to be imperceptible to us. This view also bears on interpretability: if a model succeeds by relying on human-imperceptible features, there may be little in its decisions for a human to understand.

3.3 Pertinence of Adversarial Examples as a Subject Area

The two preceding subsections shed an interesting light on adversarial attacks as a field of study in the domain of cyber security. In the first, we asked whether they pose a real danger and found that little evidence can be found that adversarial attacks pose a pertinent security threat in any practical scenario. As a live security threat they have so far been marginal, dwarfed by cruder and cheaper forms of attack [apruzzese2023real; raff2023dont]. Adding in our examination of different explanations for the phenomenon, we come to the conclusion that the primary significance of adversarial examples is of *epistemic* nature. Their main impact can be seen as offering a new view on the inherent vulnerabilities of machine learning systems; shaking up pre-existing beliefs. In other words, their importance lies less in what they let an attacker do than in what they reveal about the systems they target.

What they reveal is considerable. Before the discovery and systematic study of adversarial examples, machine-learning models were evaluated almost exclusively on accuracy, and robustness was expected only implicitly [biggio2018wild; hendrycks2019benchmarking]. Adversarial examples showed that high test accuracy does not imply that a model has grasped a learning task the way a human would: a classifier will confidently mislabel an input that is, to a human observer, semantically unchanged. This was the original observation that made the phenomenon of adversarial attacks so remarkable. This forced a more intricate account of how machine learners operate—from Goodfellow et al.’s high-dimensional linearity to Ilyas et al.’s reframing of adversarial features as genuine but human-imperceptible signal [goodfellow2014explaining; ilyas2019adversarial]—and, with it, a redefinition of *robustness* from an assumed by-product of accuracy into a distinct property that must be explicitly demanded, engineered, and evaluated. This shift is not merely academic: as machine learning is embedded in autonomous vehicles, border control, and surveillance, physical adversarial attacks make plain that robustness cannot be taken for granted in exactly those deployments where its absence would be most costly.

Viewed from a distance, the lesson generalises well beyond the field. When we trust a simple mathematical proxy to stand in for a messy real-world objective—as an overriding focus on accuracy does for machine learners—we are bound to be disappointed, and the resulting mismatch can surface suddenly and in unexpected ways. The contribution of adversarial examples is to have made that mismatch—between human and machine notions of similarity and understanding—impossible to ignore.

4 Conclusion

This report has given a broad overview of adversarial attacks in machine learning. After defining adversarial examples and organising the field along a clear taxonomy, we surveyed the most influential methods. We then stepped back to consider adversarial attacks in the real and physical world, the competing attempts to explain why they exist, and what their study has contributed to the field.

Our central conclusion is that the significance of adversarial examples is primarily *epistemic*. As a practical security threat they have so far been marginal, their lasting impact lies in what they reveal about machine learning itself. They demonstrated that high test accuracy does not mean a human-like grasp of a task, exposed a fundamental mismatch between human and machine perception, and turned *robustness* from an implicit assumption into a property that must be explicitly defined, demanded, and asserted.

Statement of Contributions

Bjarne Thal: Introduction (1), Definition of adversarial examples (2.1), Taxonomy (2.2), L-BFGS, DeepFool, Carlini-Wagner, Discussion (3), Conclusion (4)

Marius Wrobel: FGSM, Variants of FGSM, PGD, Transfer Attacks (Jacobian-based, Ensemble Attacks, Momentum), ZOO, Notable Poisoning Attacks (2.4)

Both of us conducted literature search.

Eidesstattliche Versicherung

Statutory Declaration in Lieu of an Oath

Name, Vorname/Last Name, First Name

Matrikelnummer (freiwillige Angabe)

Matriculation No. (optional)

Ich versichere hiermit an Eides Statt, dass ich die vorliegende Arbeit/Bachelorarbeit/
Masterarbeit* mit dem Titel

I hereby declare in lieu of an oath that I have completed the present paper/Bachelor thesis/Master thesis* entitled

selbstständig und ohne unzulässige fremde Hilfe (insbes. akademisches Ghostwriting)
erbracht habe. Ich habe keine anderen als die angegebenen Quellen und Hilfsmittel benutzt.
Für den Fall, dass die Arbeit zusätzlich auf einem Datenträger eingereicht wird, erkläre ich,
dass die schriftliche und die elektronische Form vollständig übereinstimmen. Die Arbeit hat in
gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

independently and without illegitimate assistance from third parties (such as academic ghostwriters). I have used no other than
the specified sources and aids. In case that the thesis is additionally submitted in an electronic format, I declare that the written
and electronic versions are fully identical. The thesis has not been submitted to any examination body in this, or similar, form.

Ort, Datum/City, Date

Unterschrift/Signature

*Nichtzutreffendes bitte streichen

*Please delete as appropriate

Belehrung:

Official Notification:

§ 156 StGB: Falsche Versicherung an Eides Statt

Wer vor einer zur Abnahme einer Versicherung an Eides Statt zuständigen Behörde eine solche Versicherung
falsch abgibt oder unter Berufung auf eine solche Versicherung falsch aussagt, wird mit Freiheitsstrafe bis zu drei
Jahren oder mit Geldstrafe bestraft.

Para. 156 StGB (German Criminal Code): False Statutory Declarations

Whoever before a public authority competent to administer statutory declarations falsely makes such a declaration or falsely
testifies while referring to such a declaration shall be liable to imprisonment not exceeding three years or a fine.

§ 161 StGB: Fahrlässiger Falscheid; fahrlässige falsche Versicherung an Eides Statt

(1) Wenn eine der in den §§ 154 bis 156 bezeichneten Handlungen aus Fahrlässigkeit begangen worden ist, so
tritt Freiheitsstrafe bis zu einem Jahr oder Geldstrafe ein.

(2) Strafflosigkeit tritt ein, wenn der Täter die falsche Angabe rechtzeitig berichtet. Die Vorschriften des § 158
Abs. 2 und 3 gelten entsprechend.

Para. 161 StGB (German Criminal Code): False Statutory Declarations Due to Negligence

(1) If a person commits one of the offences listed in sections 154 through 156 negligently the penalty shall be imprisonment not
exceeding one year or a fine.

(2) The offender shall be exempt from liability if he or she corrects their false testimony in time. The provisions of section 158 (2)
and (3) shall apply accordingly.

Die vorstehende Belehrung habe ich zur Kenntnis genommen:

I have read and understood the above official notification:

Ort, Datum/City, Date

Unterschrift/Signature